Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

AUDITAGENT

Important Notice

This Report has been generated entirely by AI and has not been manually reviewed by Nethermind's security team. It does not constitute a full security audit. Projects may not represent or imply otherwise in any public communication. All findings, observations, and recommendations may contain errors or omissions and must be independently verified by a qualified human reviewer before being acted upon. This Report should be used as a supplementary tool only, alongside professional security practices.

Code Info

Developer Scan

Scan ID 23	Date September 12, 2026
Organization RigoBlock	Repository v3-contracts
Branch fix/hype-same-block-lock	Commit Hash db2816f0...fb66dfe9

Contracts in scope

contracts/protocol/libraries/HyperliquidLib.sol

Code Statistics

Findings 3	Contracts Scanned 1	Lines of Code 111

Findings Summary

Total Findings

- High Risk (0)
- Medium Risk (3)
- Low Risk (0)
- Info (0)
- Best Practices (0)

Code Summary

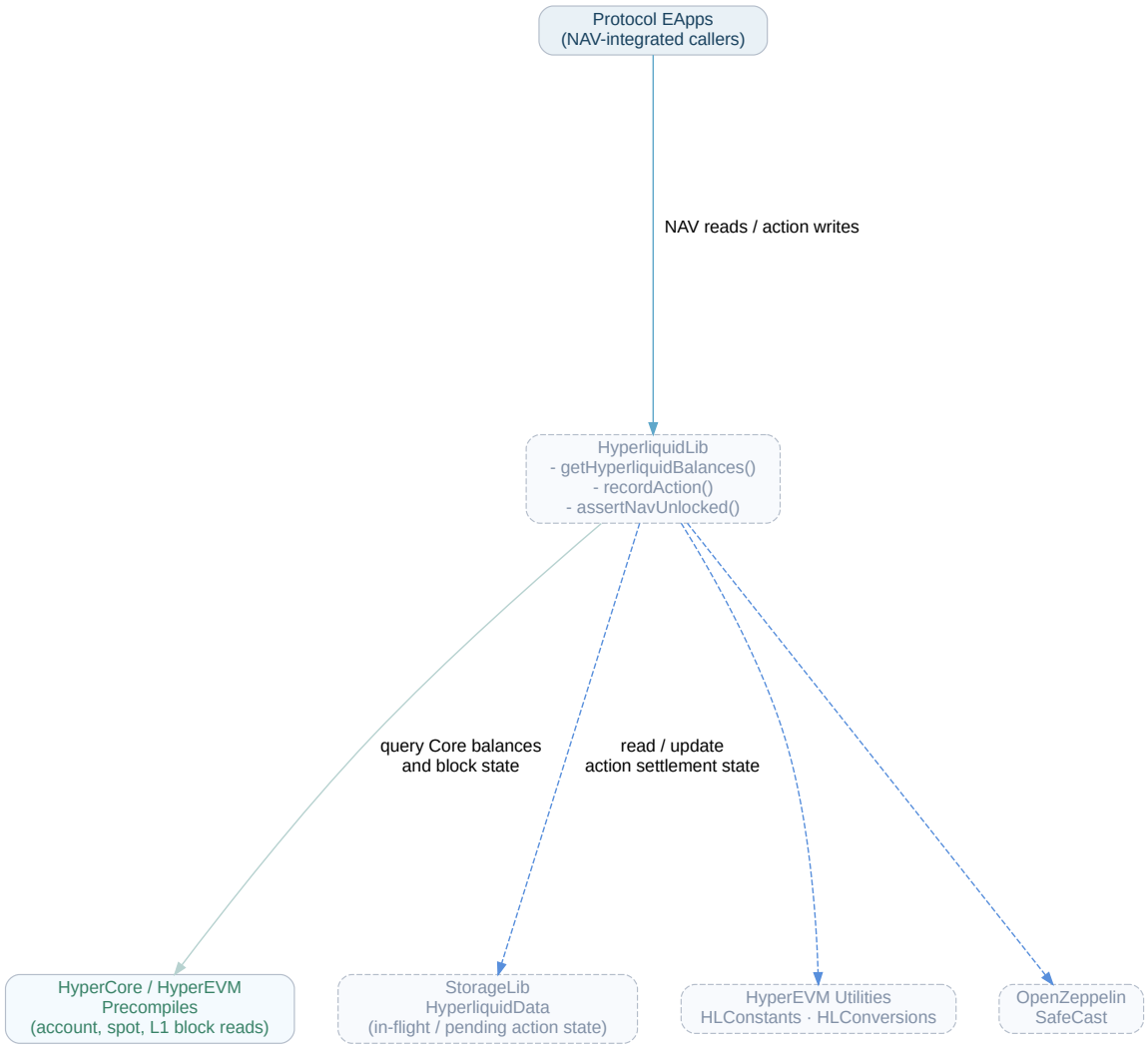
HyperliquidLib is a specialized integration library designed for the Rigoblock asset management protocol to interface with Hyperliquid on HyperEVM. It facilitates accurate Net Asset Value (NAV) accounting and state synchronization between the EVM execution environment and the underlying HyperCore layer. The library reads margin summaries from perpetual markets and token balances from spot markets via precompiled contracts, consolidating them into a unified USDC-denominated valuation for a given pool account.

To address asynchronous transitions between EVM blocks and HyperCore settlement, HyperliquidLib tracks in-flight transactions and pending spot sends using a composite block structure that pairs L1 and EVM block heights. It includes balance adjustments within the active block to prevent miscalculations and enforces a settlement time window to prevent stale NAV computations while cross-layer actions are settling.

Main Entry Points

Because HyperliquidLib is an internal library providing core math, precompile integration, and accounting helpers for other protocol contracts, it does not expose any direct public or external state-modifying functions to external actors.

Code Diagram



1 of 3 Findings

contracts/protocol/libraries/HyperliquidLib.sol

Unrecorded marketable orders can leave NAV writable against stale HyperCore account value

Medium Risk

`HyperliquidLib` makes settlement locking entirely dependent on the pool having previously called `recordAction`. The lock state is represented only by `HyperliquidData.lastActionCompositeBlock` and `lastActionTimestamp`, both of which are updated exclusively in `recordAction`.

```
function recordAction(int256 amount, bool isSpotSend) internal returns (uint64
pendingBefore) {
    HyperliquidData storage data = StorageLib.hyperliquidData();
    uint256 compositeBlock = _composeBlockNumber();
    // ...
    data.lastActionTimestamp = uint48(block.timestamp);
    // ...
}

function assertNavUnlocked() internal view {
    HyperliquidData memory data = StorageLib.hyperliquidData();
    if (data.lastActionCompositeBlock == 0) return;
    if (data.lastActionCompositeBlock == _composeBlockNumber()) return;
    require(block.timestamp >= data.lastActionTimestamp + _SETTLEMENT_WINDOW, NavLocked());
}
```

The supplied integration context states that marketable limit orders and cancels do not record a Hyperliquid cross-layer action. Consequently, a marketable order that is accepted by CoreWriter can change the pool's HyperCore account value, including through execution fees, realized PnL, or an immediate fill, without updating `lastActionCompositeBlock` or `lastActionTimestamp`.

Until the HyperEVM margin-summary precompile exposes the resulting Core state, `getHyperliquidBalances` can return the pre-trade `accountValue`. Unlike recorded EVM-to-Core deposits, there is no same-block adjustment for the order-induced value change, and unlike recorded deposits or spot sends, there is no post-block settlement lock. A NAV-writing pool action can therefore use an overstated or understated USDC NAV. Depending on the trade outcome, a share holder can redeem at an overstated NAV after a loss or a depositor can mint shares at an understated NAV after a gain, transferring value to or from other pool shareholders when the delayed Core state becomes visible.

```
int256 perpValue = int256(
    PrecompileLib.accountMarginSummary(HLConstants.DEFAULT_PERP_DEX, account).accountValue
);

int256 totalUsdcValue = perpValue + spotValue;

bool recentAction = _hasRecentAction();
if (recentAction) {
    totalUsdcValue += StorageLib.hyperliquidData().inFlightAmount;
}
```

This calculation only compensates a prior non-spot action that was explicitly recorded. A marketable order that bypasses `recordAction` has neither an in-flight adjustment nor a settlement delay, despite being capable of changing `perpValue` asynchronously.

Severity Note:
- That marketable limit orders and cancels skip recordAction is taken from the supplied integration context, not from a call site in this file.
- Material impact assumes the precompile lag (typically same EVM block after CoreWriter) overlaps a NAV write and that the order's PnL/fees are large enough to matter versus pool NAV.

2 of 3 Findings

contracts/protocol/libraries/HyperliquidLib.sol

A later Core action re-enables same-block NAV reads while an earlier action remains unsettled

Medium Risk

`recordAction` retains an in-flight adjustment only for the most recently recorded composite block. When a new EVM/composite block records another Core action, it unconditionally discards the prior block's `inFlightAmount` and replaces `lastActionCompositeBlock` with the new block:

```
if (data.lastActionCompositeBlock != compositeBlock) {
    data.inFlightAmount = 0;
    data.pendingSpotSend = 0;
    data.lastActionCompositeBlock = compositeBlock;
}
```

The settlement guard then immediately permits NAV-sensitive operations in the new block, solely because the most recent action was recorded in that same block:

```
if (data.lastActionCompositeBlock == _composeBlockNumber()) return;
```

This creates an unsafe sequence if an earlier non-spot action has not yet appeared in the Core reader precompiles. For example, assume action A deposits 60 USDC from HyperEVM to Core in composite block N. The pool's EVM USDC balance is reduced immediately, while `inFlightAmount` temporarily restores the missing Core value for same-block NAV reads. If the Core reader is still stale in block N+1, an authorized operator can record a second Core action B in N+1. Recording B clears the 60-USDC adjustment for A, stores only B's adjustment, and causes `assertNavUnlocked()` to return through its same-block exception.

`getHyperliquidBalances` then adds only the adjustment for B:

```
bool recentAction = _hasRecentAction();
if (recentAction) {
    totalUsdcValue += StorageLib.hyperliquidData().inFlightAmount;
}
```

The resulting NAV omits A even though the corresponding USDC has already left the pool's EVM balance. A mint, burn, share-price update, or other NAV-sensitive action executed in block N+1 can therefore use materially understated or otherwise stale pool value. The intended post-block settlement lock is bypassed before its time-based condition is evaluated.

Severity Note:
- Material harm assumes HyperCore precompiles can lag beyond one EVM block, which the integration documents as an edge case rather than proving it cannot happen.
- A second authorized Core action in a later composite block is required; without it assertNavUnlocked still enforces the 16-second window.

3 of 3 Findings

contracts/protocol/libraries/HyperliquidLib.sol

The fixed 16-second settlement lock can expire while Core reader balances are still stale

Medium Risk

The post-block settlement guard is based exclusively on elapsed EVM timestamp time. Once the composite block changes, the same-block correction is no longer included by `getHyperliquidBalances`; after 16 seconds, the guard permits NAV-sensitive execution without checking whether the relevant Core precompile values now include the recorded action:

```
function assertNavUnlocked() internal view {
    HyperliquidData memory data = StorageLib.hyperliquidData();
    if (data.lastActionCompositeBlock == 0) return;
    if (data.lastActionCompositeBlock == _composeBlockNumber()) return;
    require(block.timestamp >= data.lastActionTimestamp + _SETTLEMENT_WINDOW, NavLocked());
}
```

The in-flight correction is separately limited to exact composite-block equality:

```
function _hasRecentAction() private view returns (bool) {
    uint256 lastComposite = StorageLib.hyperliquidData().lastActionCompositeBlock;
    return lastComposite != 0 && lastComposite == _composeBlockNumber();
}
```

Consequently, an EVM-to-Core deposit can be absent from both the pool's EVM USDC balance and the raw Core reader response after the composite block changes. During the first 16 seconds this is blocked, but once `block.timestamp >= lastActionTimestamp + 16 seconds`, the library accepts the stale raw reader state regardless of whether HyperCore has actually processed the action.

The supplied integration context explicitly identifies delayed CoreWriter processing, sequencing delays, and HyperEVM big-block operation as conditions under which a Core reader response can remain stale for longer than the selected settlement interval. Under those conditions, a subsequent mint can receive shares at an understated NAV after a pending EVM-to-Core deposit, while the opposite accounting direction can cause redemptions or other NAV validations to use an overstated value. The discrepancy can be as large as the unsettled transfer amount rather than merely a rounding difference.

Severity Note:
- Treating Core reader staleness beyond lastActionTimestamp + 16s as reachable depends on integration/sequencing context, not on a proof inside this library that settlement can exceed the window.

Disclaimer

No Guarantee of Accuracy or Completeness. Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

This Report is not a security audit. This Report has been generated automatically by AuditAgent, an AI-powered automated code scanning tool. It does not constitute, and must not be represented or relied upon as constituting, a full or comprehensive security audit conducted by Nethermind or any of its personnel. A formal security audit involves manual review by experienced human auditors and is a materially different service. If you require a full security audit, please contact Nethermind's security audit team at auditagent@nethermind.io

Use of Nethermind's name. "Nethermind" is a trade mark of Demerzel Solutions Limited. Use of this Report does not authorise you to represent that your code or smart contract has been "audited by Nethermind" or to use any similar phrase. Any such representation would be false, misleading, and an infringement of Nethermind's intellectual property rights.]
No Endorsement or Security Guarantee. Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

Disclaimer. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.